



A Linear Gatekeeper that turns a founder's answers into a Business Suit

Capstone Project — Developers Institute

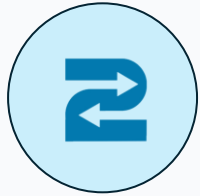
Ariel Zinger · AUG 2026

**Validate first
Build later**

Confidential

INTRODUCTION

Why I built biz2code



Project Introduction:

biz2code is a "Linear Gatekeeper" web application that guides the users through a fixed, sequence of structured questions and automatically generates a complete Business Suite

The motivation:



Product Manager Insight

The Friction in AI Validation

- ⚠️ **High Risk & Effort:** AI-driven business validation requires constant manual guidance and fixes for missing data.
- ⚠️ **Ideation Bottleneck:** Flawed early validation discourages viable concepts or gives false confidence to weak ones.
- ⚠️ **Hackathon Experience:** Personal experience exposed the inefficiency of iterative prompting without structure.



Project Introduction

The "Linear Gatekeeper" Web App

- ✓ **Front-Loads Hard Decisions:** Replaces endless trial-and-error with a disciplined, fixed intake process.
- ✓ **Structured Sequence:** Guides users step-by-step through mandatory, gatekept business questions.
- ✓ **Automated Business Suite:** Generates a complete, accurate validation package with zero reiteration needed.

PROJECT OVERVIEW

What biz2code is



Core Concept

Business Suite generator built on a *Linear Gatekeeper* with fixed sequential questions and rigid per-phase data mapping.



Target Market

Purpose-built for *Developers, Product Managers, Marketers, and Early-Stage Founders* seeking rapid validation.



Strategic Value

Enables rigorous *idea validation* without incurring the high cost or complexity of hiring full-time business analysts.

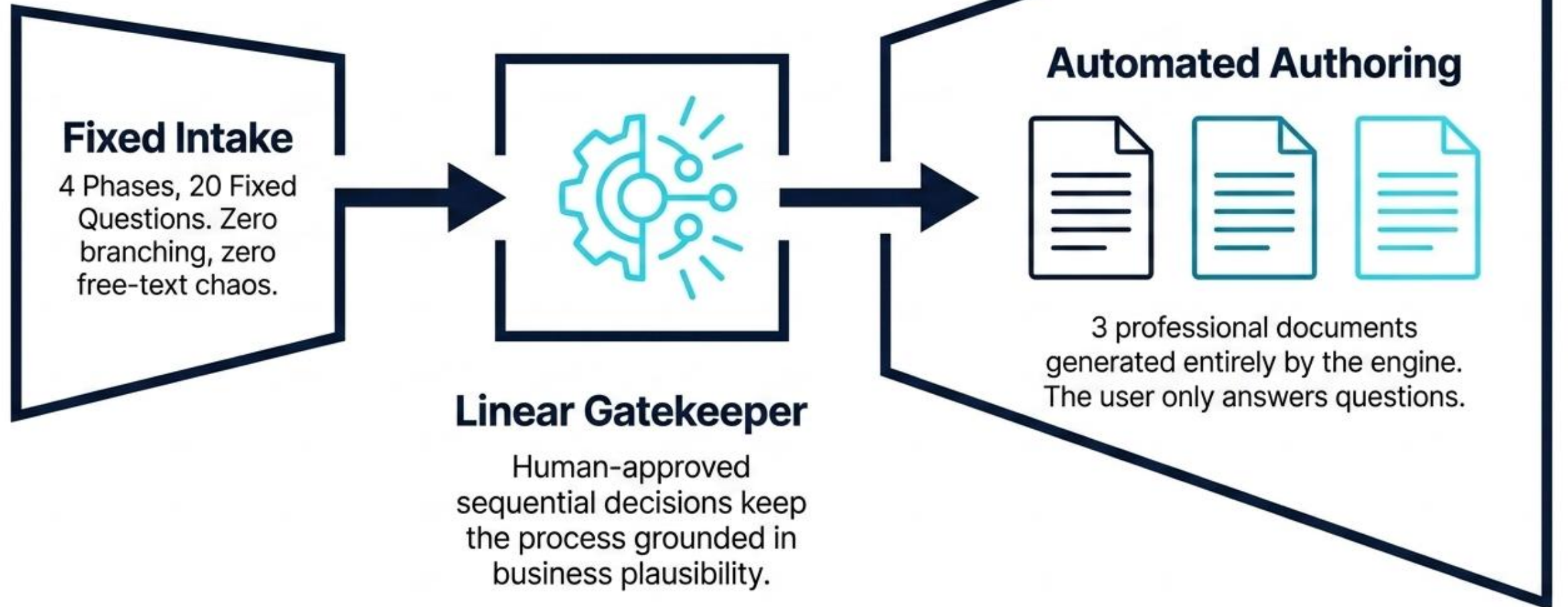


The Commercial AI Reliability Gap

Risks in Unguided AI Validation

- ⚠️ **Sycophancy Bias:** Standard LLMs tell users what they want to hear, falsely validating weak business concepts.
- ⚠️ **Answer Randomness:** Unbounded prompts produce inconsistent document formats and erratic structure.
- ⚠️ **Data Hallucinations:** Commercial AI invents quantitative metrics and financials when critical inputs are missing.
- ⚠️ **Built-in bias:** AI models inherit hidden bias directly from their training data, human prejudices, sampling gaps, and systemically slanted patterns.

The biz2code Approach: Constrain to Create



Architectural Advantage & Deliverables

Dimension	Generic Commercial AI	biz2code Architecture
Intake Workflow	⚠ Free-text prompt vulnerability	✔ Fixed Phases / Rigid Gatekept Steps
Document Structure	Inconsistent ad-hoc output	Fixed Business Suite Templates
Financial Logic	⚠ Hallucinated numbers vulnerability	✔ Deterministic TypeScript Math Engine
Data Auditability	Opaque confidence tiers	Flagged estimates & missing data warnings



Standardized Quality

Adheres strictly to industry-standard Business Plan, MRD, and PRD frameworks.



Rapid Iteration

Generates ready-to-present documentation in a single session with zero prompt fatigue.



Verifiable Math

100% auditable formulas ensure pitch deck metrics stand up to rigorous scrutiny.

POTENTIAL USERS

Who it's for - Target audience



Founders

Need pre-seed documents fast, without the expense of hiring consultants or analysts.



Product Managers

Require structured starting points for Product and Market Requirements, skipping the blank templates.



Indie Developers

Solo builders forced to wear the business hat, requiring structured guidance to validate ideas before writing code.

How the project caters to their needs:

- **No business background required.** The fixed questions bank carries the business-analysis burden , so the user only need to answer plain questions, regardless of how much business experience they bring in.
- **Consistent, standardized output.** Every user gets the same recognized business document structure.

From Idea to Production

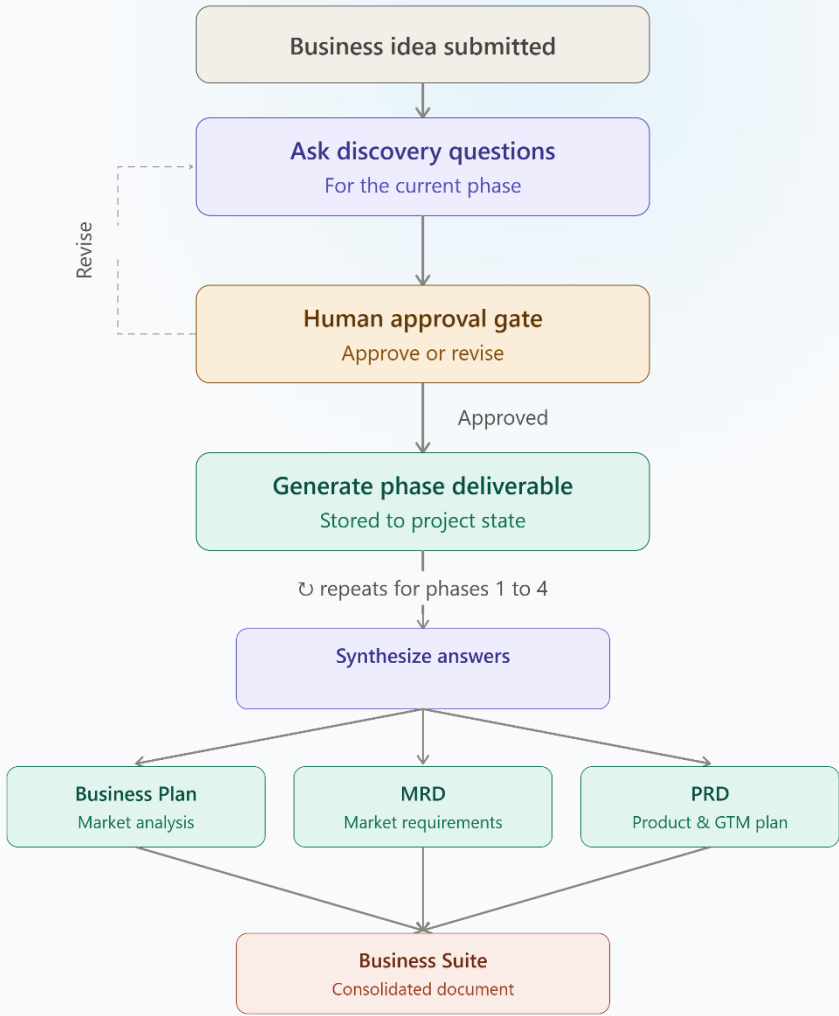
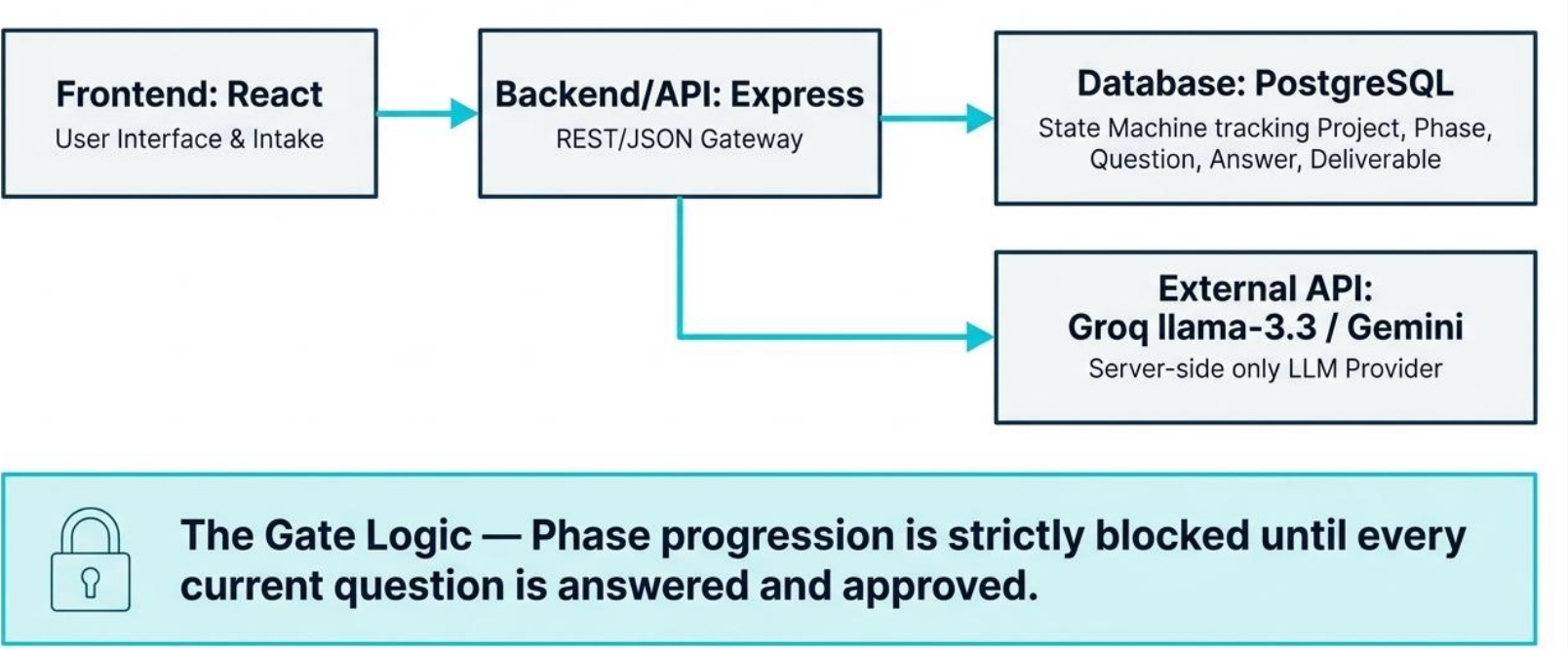
- 1 Seed Project
- 20 Guided Questions
- Live Generation: Business Plan, MRD, and PRD

Note: Seed project ships pre-filled to eliminate LLM latency during the live run.

LIVE DEMO



Under the hood: Tech Stack & State Model



PROJECT CHALLENGES

Challenges & Direct Solutions

Mapping key architectural and process hurdles directly to their resolutions

⚠ CHALLENGES FACED (WHAT GOT HARD)

01 Automation model discipline

The line between "user answers, engine writes" kept blurring during design and had to be corrected more than once.

ADR
→

02 Scope discipline

Redux, Feature-Sliced Design, and live deployment were all seriously considered, then deliberately cut due to time constraints for the MVP.

LOG
→

03 File delivery reliability

An early generation run wrote files without surfacing all of them — the kind of silent failure the project is meant to catch.

CI
→

✅ SOLUTIONS (HOW ADDRESSED)

Wrote an ADR for every call

Each non-obvious decision has a recorded trail, not memory.

Treated scope creep as a stop condition

Deferred features were named and logged, not quietly built anyway.

Added a CI-enforced data guardrail

A CI validator (benchmarks.json) stops placeholder or fabricated figures from reaching a document.

FUTURE STEPS

What's next

Next iteration

Natural extensions once the MVP is proven and graded.



Dynamic Project Ledger - a live-updating record of project state beyond the static intake.



Financial Dashboard - visual/interactive view of financial figures, beyond the DOCX output.



Solution Architecture Document - a fourth generated document, on top of the Business Suite three.



Handoff.md - a structured handoff artifact for passing the project to another builder or team.



Conditional (branching) questions

Adapt the question flow based on earlier answers.



Multi-project support

Manage more than one venture from a single account.



Live deployment

Ship directly from the platform, beyond local-only builds.

SUMMARY

Key achievements and outcomes



A working Linear Gatekeeper

A fixed intake of 23 questions produces three DOCX documents — and a verdict on whether to proceed.



A real trust boundary

Deterministic calculation kept strictly separate from narrative generation. Run it twice, get identical figures.



A documented decision trail

16 ADRs, amended when the build contradicted them rather than left to drift.



A data guardrail that holds

A validator stops a placeholder ever carrying a number — and it survived a live prompt-injection attack.



Full-stack build under real constraints

React, TypeScript and raw PostgreSQL, verified against a real database and a real browser.

Validate first, Build later.